
大数据与人工智能
自动化学院

CHAPTER3-2： PYTORCH的动态图、自动求导及逻辑回归

3.2-1 导言

主要内容：

- 计算图与Pytorch的动态图机制（计算图的概念，动态图与静态图的差异和搭建过程）
- Pytorch的自动求导机制
- 基于前面所学搭建一个逻辑回归模型
- 总结梳理

3.2-2. 计算图与PYTORCH的动态图机制

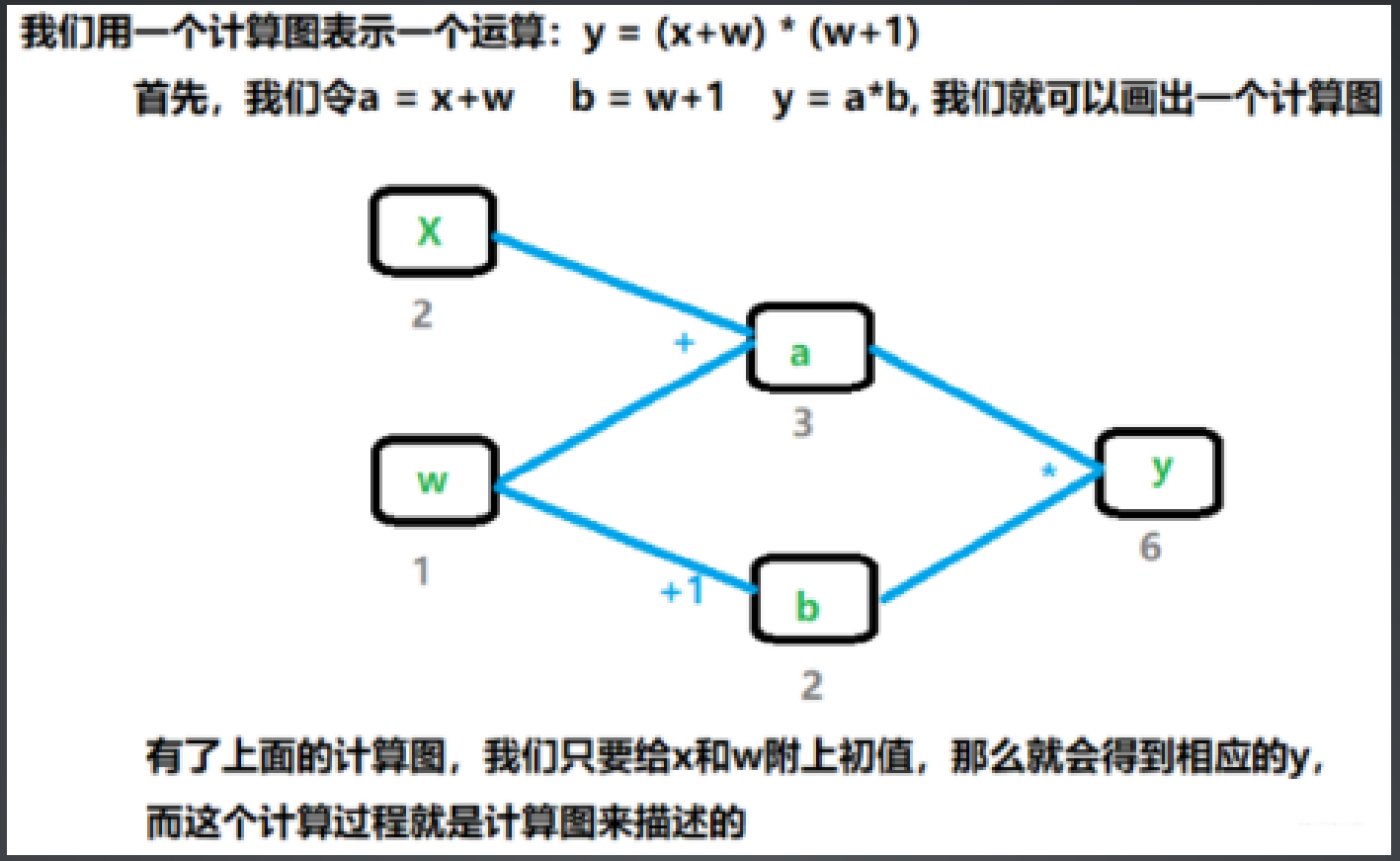
3.2-2.1计算图

前面已经整理了张量的一系列操作

深度学习， 其实就是对各种张量进行操作

- 随着操作的种类和数量的增大，会导致各种想不到的问题，比如多个操作之间该并行还是顺序执行，底层的设备如何协同，如何避免冗余的操作等。
- 而计算图就是为了解决这些问题而产生的， 那么什么是计算图呢?

- 主要有两个因素： 节点和边。 其中节点表示数据，如向量，矩阵，张量。边表示运算，如加减乘除，卷积等。
- 下面我们看一下具体这东西是什么样子：



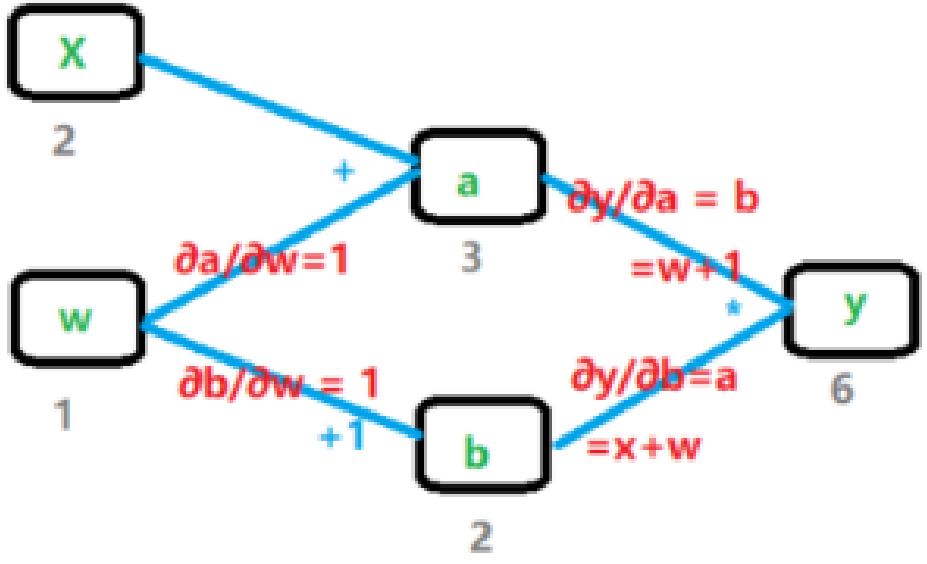
使用计算图的好处不仅让计算看起来更加简洁，还有个更大的优势就是让梯度求导也变得更加方便。

下面我们看看y对w进行求导的过程：

我们用一个计算图表示一个运算：y = (x+w) * (w+1)

首先，我们令a = x+w　　b = w+1　　y = a*b，我们就可以画出一个计算图

我们看一下y对w求导的过程：



所以，y对w的导数：

$$\partial y/\partial w = \partial y/\partial a * \partial a/\partial w + \partial y/\partial b * \partial b/\partial w$$

$$= b * 1 + a * 1$$

$$= (w+1) + (x+w) = 2*w + x + 1 = 2*1 + 2 + 1 = 5$$

y对w求导，就是从计算图中找到所有y到w的路径。把各个路径的导数进行求和。

我们通过程序来验证一下：

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)
y = torch.mul(a, b)

y.backward()
print(w.grad)    # tensor([5.])
```

下面，我们基于这个计算图来说几个张量里面重要的属性：

1. 叶子节点这个属性(还记得张量的属性里面有一个is_leaf吗）：

- 叶子节点： 用户创建的节点， 比如上面的x和w。叶子节点是非常关键的，在上面的正向计算和反向计算中，其实都是依赖于我们叶子节点进行计算的。
- is_leaf: 指示张量是否是叶子节点。
- 为什么要设置叶子节点这个概念？ 主要是为了节省内存
- 因为在反向传播完了之后，非叶子节点的梯度是默认被释放掉的。

我们可以根据上面的那个计算过程，来看看w，x, a, b, y的is_leaf属性，和它们各自的梯度情况：

```
#查看叶子结点
print("is_leaf:\n", w.is_leaf, x.is_leaf, a.is_leaf, b
#查看梯度，  默认是只保留叶子节点的梯度的
print("gradient:\n", w.grad, x.grad, a.grad, b.grad, y

## 结果:
is_leaf:
 True True False False False
gradient:
 tensor([5.]) tensor([2.]) None None None
```

- 我们可以发现， 只有w, x的is_leaf属性是True， 说明这俩是叶子节点。
- gradient上，也只有叶子节点的梯度被保留了下来， a, b, y的梯度都默认释放了，所以是空。
- 但是我们如果想用这里面的某个梯度呢？ 比如我想保留a的梯度， 那么可以使用`retain_grad()`方法。 就是在执行反向传播之前， 执行一行代码:`a.retain_grad()`即可。

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
a.retain_grad()
b = torch.add(w, 1)
y = torch.mul(a, b)

y.backward()
#查看梯度， 默认是只保留叶子节点的梯度的
print("gradient:\n", w.grad, x.grad, a.grad, b.grad, y

## 结果： a的梯度被保留了下来
```

2. grad_fn: 记录创建该张量时所用的方法（函数），记录这个方法主要**用于梯度的求导**。要不然pytorch求导的时候怎么知道具体是啥运算？

- 比如两数相加，或者两数相乘，这样反向计算梯度的时候才能使用相应的法则求变量的梯度。
- 当然我们其实只需要知道grad_fn是用于反向传播即可。

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
a.retain_grad()
b = torch.add(w, 1)
y = torch.mul(a, b)

y.backward()

# 查看 grad_fn    这个表示怎么得到的
print("grad_fn:\n", w.grad_fn, x.grad_fn, a.grad_fn, b
```

3.2-2.2 PYTORCH的动态图机制

根据计算图的搭建方式，可以将计算图分为动态图和静态图。

- 静态图： 先搭建图，后运算。高效，不灵活（TensorFlow）
- 动态图： 运算与搭建同时进行。灵活，易调节（Pytorch）

这个就类似于旅游的时候你参加旅行团还是自己去旅游一样

- 参加旅行团的这种就类似于静态图，游览的路线和流程都已经确定，跟着走就行了。
- TensorFlow1.x，是先把图搭建好，然后开启一个会话， 在那里面才开始喂入数据进行流动计算， 在这个过程中，张量就会根据搭建好的图进行计算。
- TensorFlow2.x更新之后也采用了动态图机制。

- 而自己去旅游这个就类似于动态图，游览的路线和流程都不确定，想去哪去哪，随时调整。
- Pytorch就是边建图边执行，比较灵活， 有错误可以随时改，也更接近我们一般的想法。

依然是上面的例子：

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)
y = torch.mul(a, b)   # y=(w+x)*(w+1)
print(y)     # tensor([6.], grad_fn=<MulBackward0>)
```

这里会发现直接就算出了y的结果，这说明上面的每一步都进行了计算。

PyTorch 在内部会构建一个计算图 (Computation Graph)。对于 `y1 = (w + x) * (w + 1)`，这个图可以分解为以下几步：

1. **中间步骤 1**: `a = w + x`
2. **中间步骤 2**: `b = w + 1`
3. **最终步骤**: `y1 = a * b`

当执行 `y1.backward()` 时，梯度会从 `y1` 开始，沿着这个图反向传播，每一步都乘以当地的梯度。

提问：Pytorch中的动态图机制与静态图机制的主要区别是什么？

A. 动态图在运算前先搭建图，静态图边运算边搭建
B. 动态图边运算边搭建，静态图在运算前先搭建图
C. 动态图和静态图没有区别
D. 静态图更灵活，动态图更高效

答案： B

讲评：

Pytorch采用的是动态图机制，这意味着计算图是在运算时动态生成的，允许更灵活的调试和修改。而静态图（如TensorFlow 1.x）则是在运算前先搭建好计算图，然后在会话中执行。动态图更灵活，适合实验和调试，而静态图则更高效，但不易调整

3.2-3. PYTORCH的自动求导机制

Pytorch自动求导机制使用的是torch.autograd.backward方法， 功能就是自动求取梯度。

```
torch.autograd.backward(tensors,
                        grad_tensors=None,
                        retain_graph=None,
                        create_graph=False)
```

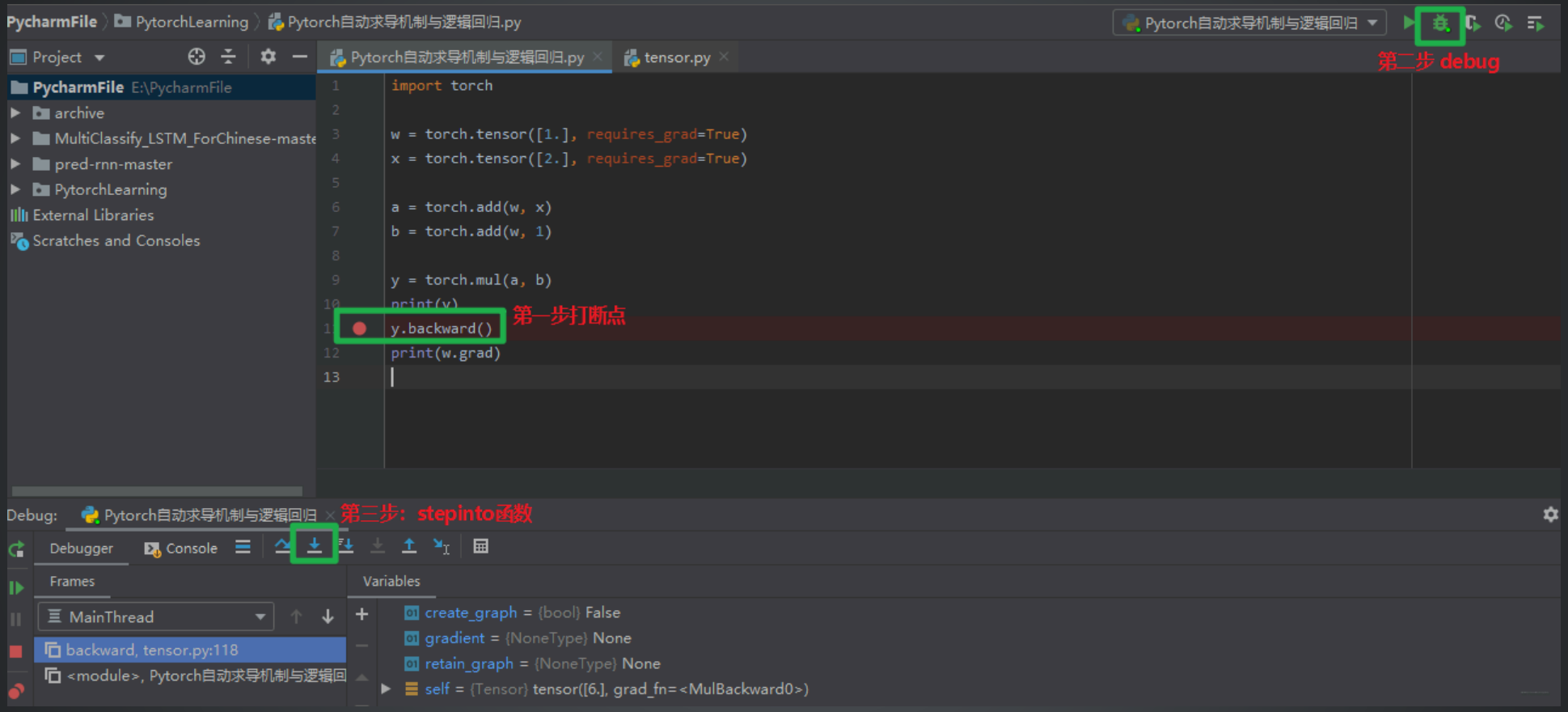
- tensors表示用于求导的张量，如loss。
- retain_graph表示保存计算图， 由于Pytorch采用了动态图机制，在每一次反向传播结束之后，计算图都会被释放掉。如果我们不想被释放，就要设置这个参数为True
- create_graph表示创建导数计算图，用于高阶求导。
- grad_tensors表示多梯度权重。如果有多个loss需要计算梯度的时候，就要设置这些loss的权重比例。

疑问

我们上面写代码的过程中并没有见过这个方法啊？ 我们当时不是直接y.backward()吗？ 哪有什么torch.autograd.backward()啊？

其实，当我们执行y.backward()的时候，背后其实是在调用后面的这个函数，

不信？ 我们来调试一下子就清楚了：

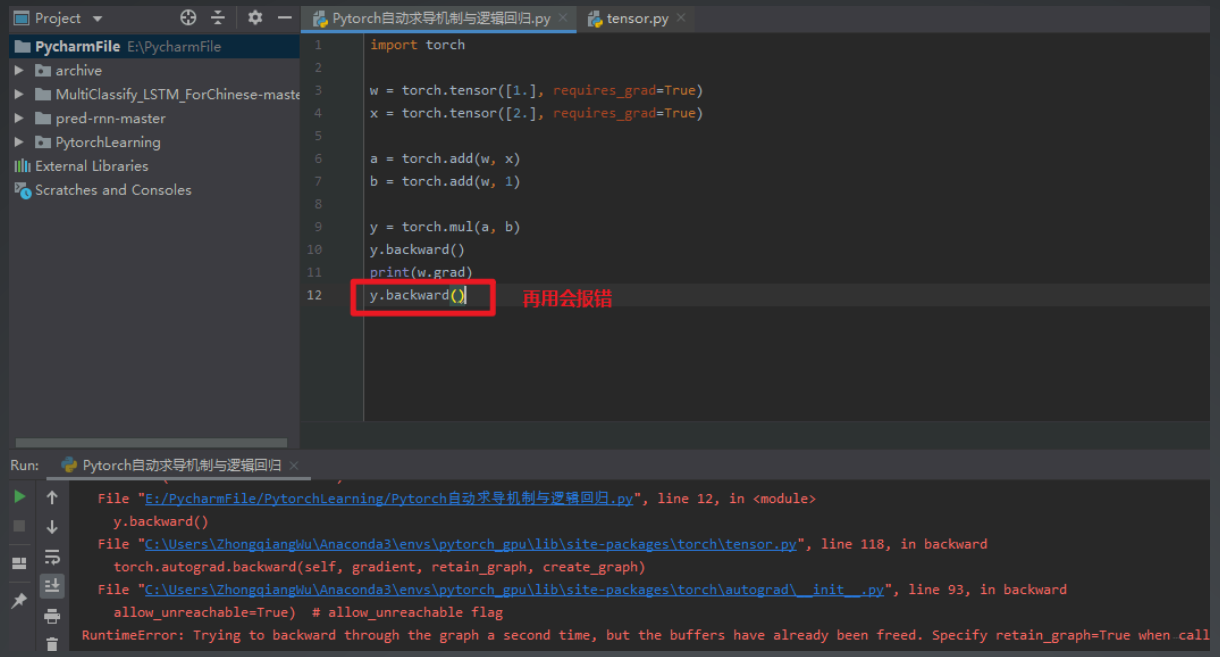


我们在这一行打断点，然后进行调试。 我们进入这个函数之后，会发现：

```
def backward(self, gradient=None, retain_graph=None, 
    torch.autograd.backward(self, gradient, retain_gra
```

这样就清楚了，这个backward函数就是在调用这个自动求导的函数。

- backward()里面有个参数叫做retain_graph, 这个是控制是否需要保留计算图的
- 默认是不保留，即一次反向传播之后，计算图就会被释放掉
- 这时候，如果再次调用y.backward， 就会报错：

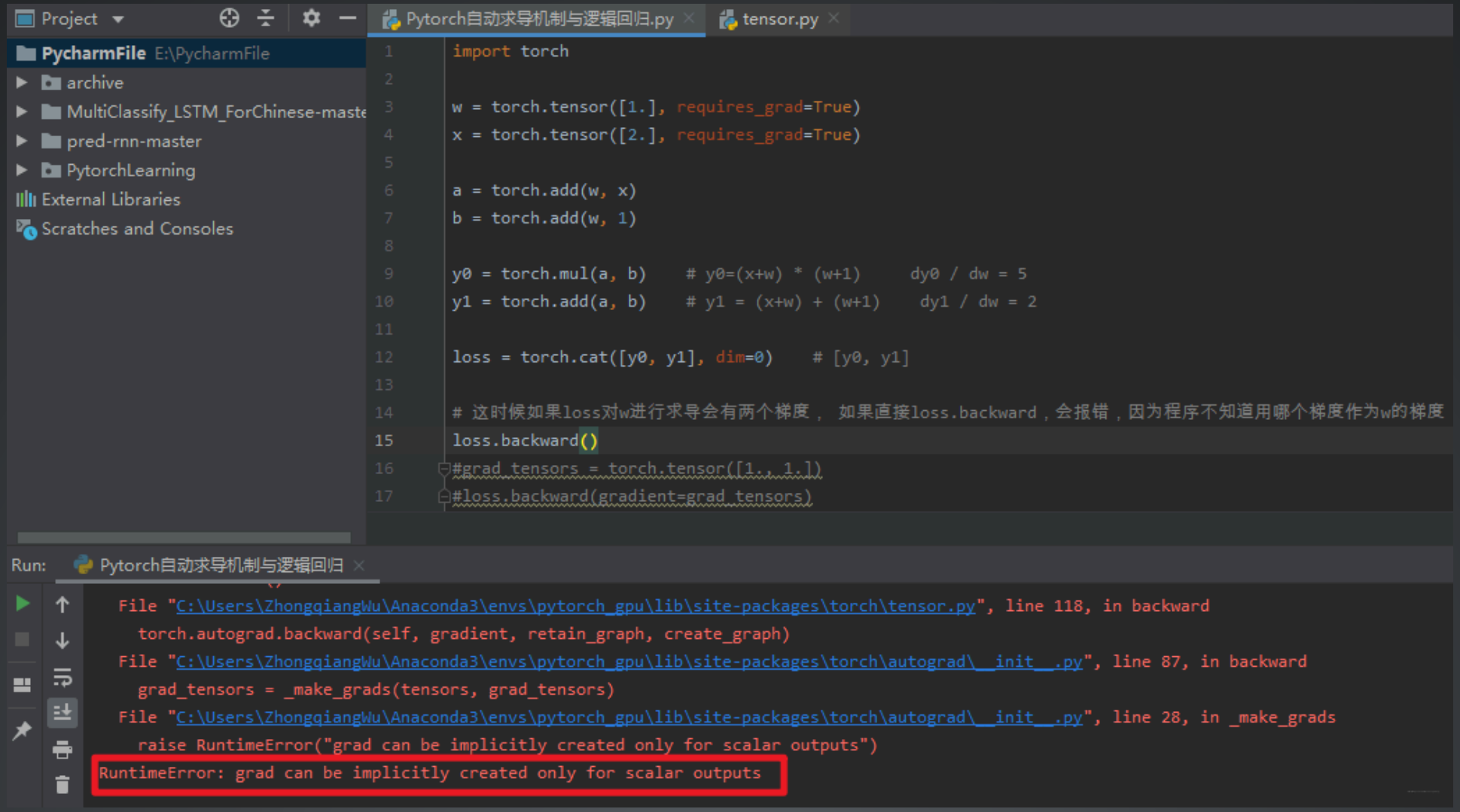


- 报错信息就是说的计算图已经释放掉了。
- 所以我们把第一次用反向传播的那个retain_graph设置为True就OK了：

```
y.backward(retain_graph=True)
```

- 这个是当有多个梯度的时候，控制梯度的权重

依然拿上面的那个举例：



上面这个过程会报错

这时候我们就需要用到gradient这个参数了

- 给两个梯度设置权重，最后得到的w的梯度就是带权重的这两个梯度之和。

```
# 为了演示gradient参数的效果，我们构建两个输出
y1 = (w + x) * (w + 1) # 第一个输出
y2 = 2 * w # 第二个输出

# 将两个输出合并成一个向量y
y = torch.stack([y1, y2])

grad_tensors = torch.tensor([1., 1.])
y.backward(gradient=grad_tensors)
print(w.grad)    #  这时候会是tensor([7.])    5+2
w.grad.zero_()

grad_tensors = torch.tensor([1., 2.])
```

- 还有一个比较常用的方法叫做：torch.autograd.grad()
- 这个方法的功能是求取梯度， 这个可以实现高阶的求导

```
torch.autograd.grad(outputs,
                    inputs,
                    grad_outputs=None,
                    retain_graph=None,
                    create_graph=False)
```

- outputs: 用于求导的张量， 如loss
- inputs: 需要梯度的张量， 如上面例子的w
- create_graph: 创建导数计算图，用于高阶求导
- retain_graph: 保存计算图
- grad_outputs: 多梯度权重

拿个例子看一下就明白了这个方法如何使用了：

```
x = torch.tensor([3.], requires_grad=True)
y = torch.pow(x, 2)    # y=x
# 一次求导
grad_1 = torch.autograd.grad(y, x, create_graph=True)
print(grad_1)    # (tensor([6.], grad_fn=<MulBackward0>

# 二次求导
grad_2 = torch.autograd.grad(grad_1[0], x)     # grad_2
print(grad_2)  # (tensor([2.]),)
```

关于Pytorch的自动求导系统要注意：

1. **梯度不自动清零**：就是每一次反向传播，梯度都会叠加上去，这个要注意，举个例子：

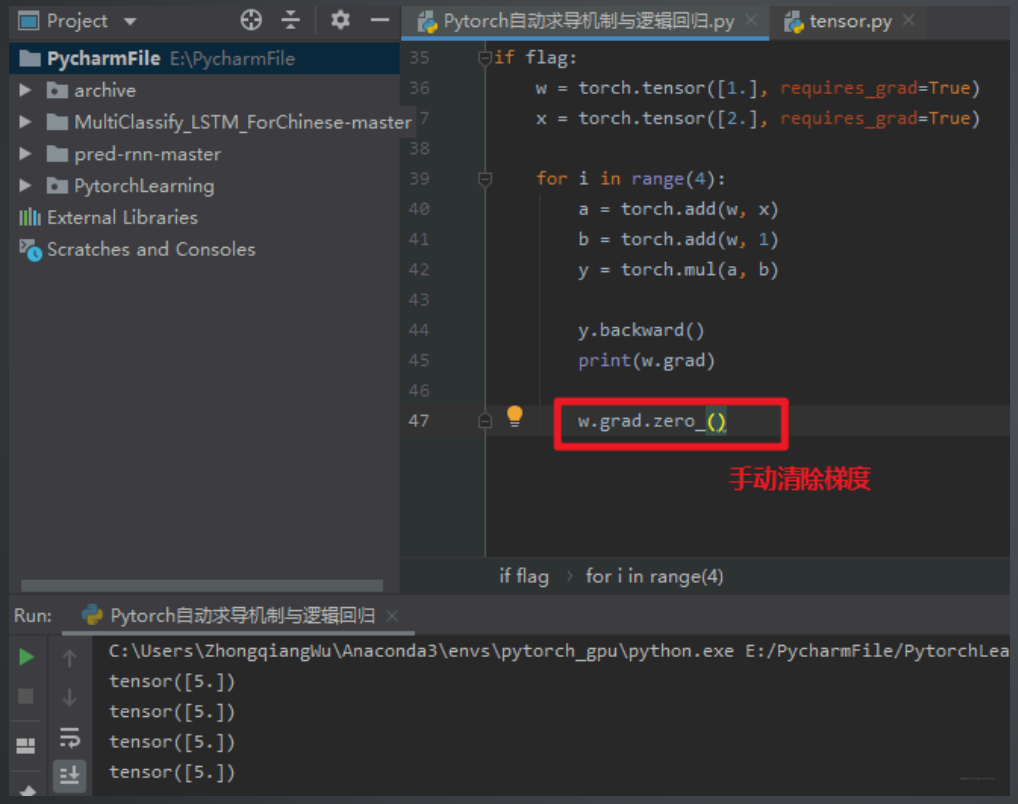
```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

for i in range(4):
    a = torch.add(w, x)
    b = torch.add(w, 1)
    y = torch.mul(a, b)

    y.backward()
    print(w.grad)

## 结果:
tensor([5.])
```

- 运行之后会发现，每次w的梯度都会累加， 执行了四次，最后是20了。
- 这样就会发生错误了，尤其是训练神经网络的时候特别注意。
- 毕竟我们肯定不是训练一次，所以每一次反向传播之后，我们**要手动的清除梯度**



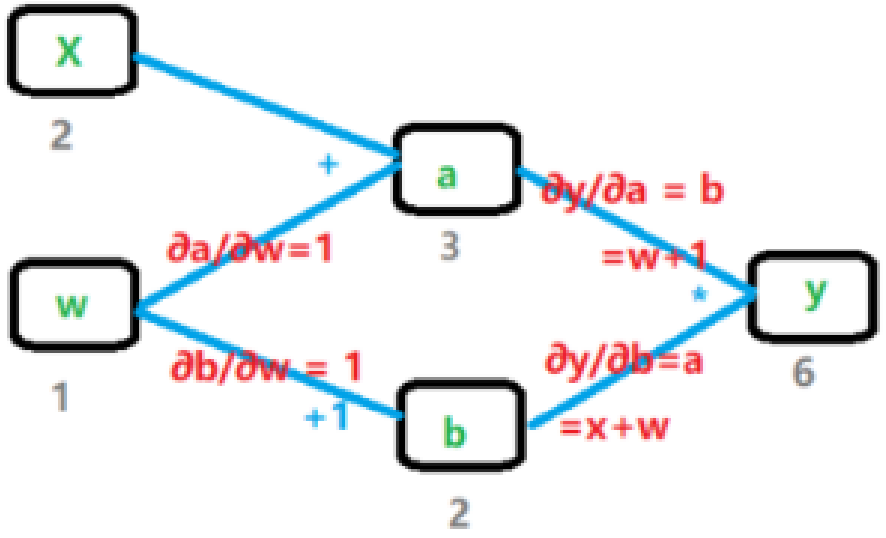
- 这里会发现个zero_()， 下划线这个代表原位操作， 后面会说。

2. 依赖于叶子节点的节点， requires_grad默认为True， 这是啥意思?

我们用一个计算图表示一个运算：y = (x+w) * (w+1)

首先，我们令a = x+w　　b = w+1　　y = a*b, 我们就可以画出一个计算图

我们看一下y对w求导的过程：



所以， y对w的导数：

$$\partial y/\partial w = \partial y/\partial a * \partial a/\partial w + \partial y/\partial b * \partial b/\partial w$$

$$= b * 1 + a * 1$$

$$= (w+1) + (x+w) = 2*w + x + 1 = 2*1 + 2 + 1 = 5$$

拿上面的计算图过来解释一下

- 依赖于叶子节点的节点，在上面图中w,x是叶子节点，而依赖于叶子节点的节点，其实这里说的就是a,b， 也就是a，b默认就是需要计算梯度的。
- 这个也好理解，因为计算w,x的梯度的时候是需要先对a, b进行求导的，要用到a, b的梯度，所以这里直接默认a, b是需要计算梯度的。 在代码中，也就是这个意思：

```
w = torch.tensor([1.], requires_grad=True)
x = torch.tensor([2.], requires_grad=True)

a = torch.add(w, x)
b = torch.add(w, 1)

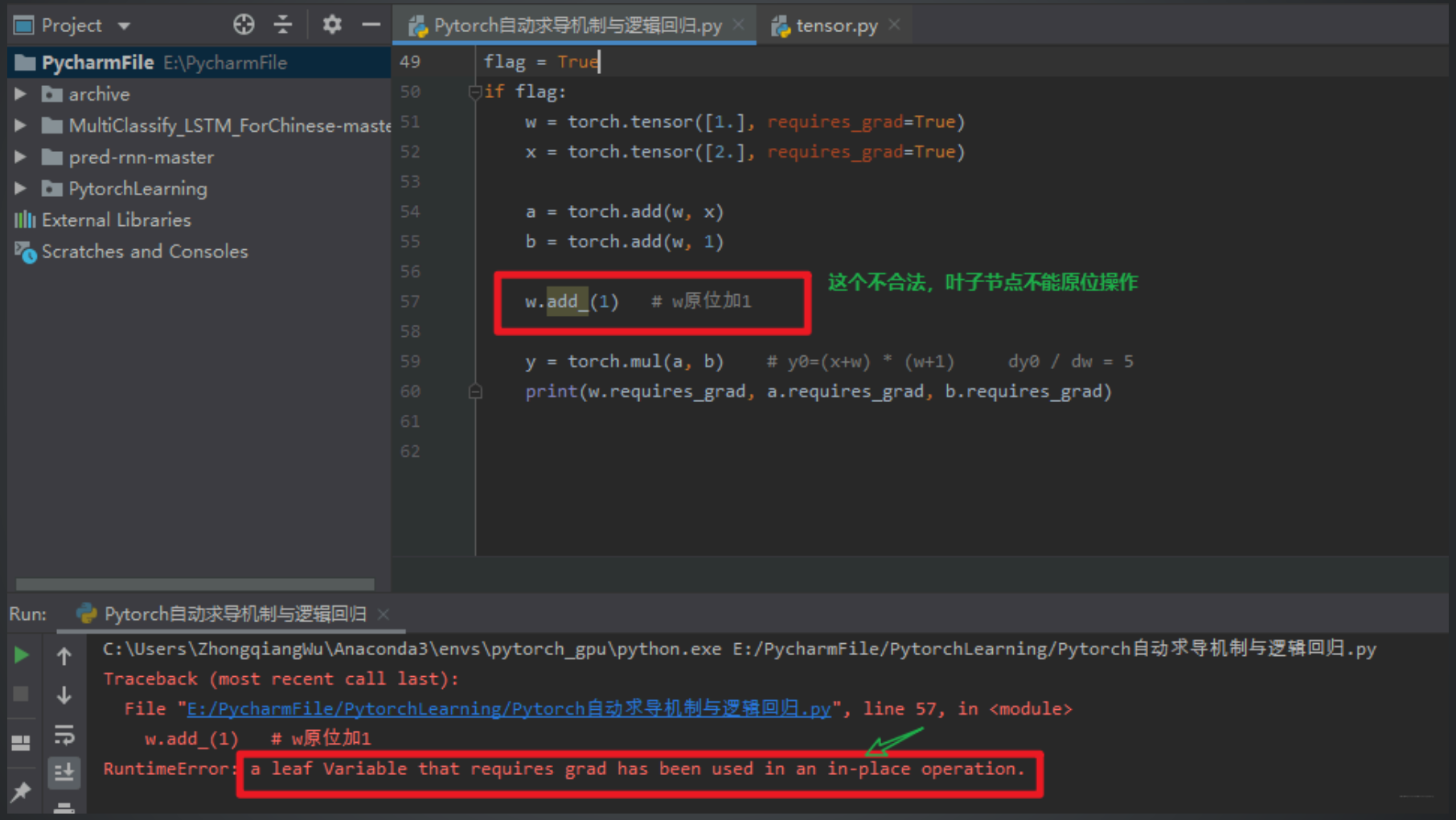
y = torch.mul(a, b)     # y0=(x+w) * (w+1)      dy0 / dw
print(w.requires_grad, a.requires_grad, b.requires_gra
```

但注意，虽然这里默认是计算梯度的，但非叶子节点的梯度默认是不保存的，也就是会被清空掉，所以如果想要保留下来，依然是需要再建立计算图之前加入a.retain_grad()这个操作。

3. 叶子节点不可执行in-place（这个in-place就是原位操作）

- 首先什么是in-place操作， 这个操作就是在原始内存当中去改变这个数据
- 非原位操作：拿a+1的例子看一下，假设a=1, 然后我执行a=a+1, 这样的话，a虽然是2，但是这两个a其实指向的对象是不一样的，原来的1所在的内存的数据并没有改变，执行a+1, **是新建了一个对象出来**， 然后改变了原来a的指向。
- 原位操作， 将数字进行原位操作之后， 这个数字就是在本身的内存当中改变的数，这时候就没有新对象建立出来。 a+=1就是一种原位操作。

- 那么为什么叶子节点不能进行原位操作呢？ 先看看叶子节点进行原位操作是怎么回事？ 下面这个报错：

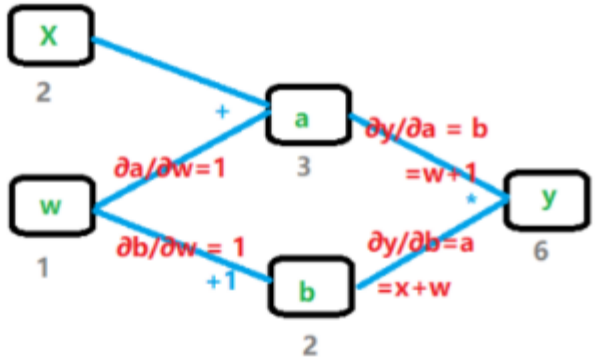


- 这是为什么呢？ 这个要从计算图求取梯度的过程来理解，依然得把计算图拿过来：

我们用一个计算图表示一个运算：y = (x+w) * (w+1)

首先，我们令a = x+w　　b = w+1　　y = a*b, 我们就可以画出一个计算图

我们看一下y对w求导的过程：



所以， y对w的导数：

$$\partial y/\partial w = \partial y/\partial a * \partial a/\partial w + \partial y/\partial b * \partial b/\partial w$$

$$= b * 1 + a * 1$$

$$= (w+1) + (x+w) = 2*w + x + 1 = 2*1 + 2 + 1 = 5$$

- 我们来看这个求取梯度的过程
- 我们要求w的梯度的时候，我们发现会先∂y/∂a=w+1, 然后∂a/∂w, 也就是说反向传播的过程的∂y/∂a就用到了w
- 这时候是怎么找到w的呢？ 其实正向传播的时候，会把w的地址给记下来
- 然后反向传播的这一步，就是根据这个地址去找w的值。
- 如果在反向传播之前，就用原位操作把这个w的值给变了，那么反向传播再拿到这个w的值的时候，就出错了。
- 所以Pytorch不允许对叶子使用原位操作

提问：在Pytorch中，为什么叶子节点的梯度默认会被保留，而非叶子节点的梯度会被释放?

答案： D

讲评：

叶子节点是用户创建的张量，反向传播时依赖这些节点来计算梯度，因此它们的梯度默认会被保留。而非叶子节点的梯度通常会被释放，以节省内存。如果需要保留非叶子节点的梯度，可以使用`retain_grad()`方法

前面已经学习了数据的载体张量，学习了如何通过前向传播搭建计算图，同时通过计算图进行梯度的求解，有了数据，计算图和梯度，我们就可以正式的训练机器学习模型了。接下来，我们就搭建一个逻辑回归模型吧。

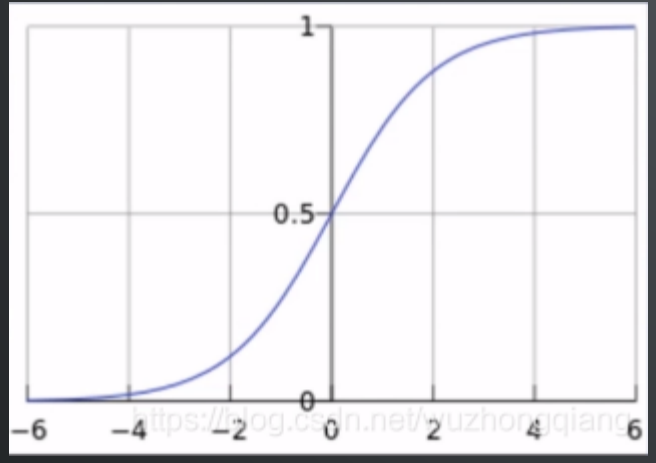
4. 逻辑回归模型

逻辑回归模型是**线性**的**二分类**模型，模型的表达式如下：

$$y = f(wx + b)$$

$$f(x) = \frac{1}{1 + e^{-x}}$$

这里的f(x)就是sigmoid函数了， 也成为了logistic函数。 还记得sigmoid函数吗?



https://blog.csdn.net/yuzhongqiang

逻辑回归模型是**线性**的**二分类**模型

为什么说是二分类的问题呢？

$$\text{class} = \begin{cases} 0, 0.5 > \text{y} \\ 1, 0.5 \leq \text{y} \end{cases}$$

我们是根据这个y的取值进行分类的，当取值小于0.5， 就判别为类别0， 大于0.5， 就判别为类别1.

那为什么称为线性呢?

我们可以对比一下线性回归和逻辑回归的区别:

- 线性回归: 自变量是 X, 因变量是 y, 关系: $y = wx + b$, 图像是一条直线。是分析自变量 x和因变量 y(标量)之间关系的方法。 注意这里的线性是针对于 w说的, 一个 w只影响一个 x。决策边界是一条直线
- 逻辑回归: 自变量是 X, 因变量是 y, 只不过这里的 y 变成了概率。 关系:

$$y = f(wx + b)$$
$$f(x) = \frac{1}{1 + e^{-x}}$$

- 图像也是一条直线。 是分析自变量 x 与因变量 y (概率）之间的关系。
- sigmoid函数在这里只是为了更好的描述分类置信度。
- 如果我们不用这个函数，其实也是可以进行二分类的，比如 $w x + b > 0$, 我们判定为1， $w x + b < 0$, 我们判定类别0， 这样其实也是可以的
- 会发现，依然是一个 w只影响一个 y。所以依然是线性的。

- 逻辑回归也叫做对数几率回归
- 首先，什么是对数几率回归，我们知道线性回归是 y = w x + b, 而如果我们把几率

$\frac{y}{1-y}$

取对数，让它等于 w x + b 就叫做对数几率回归， 即

$$\ln \frac{y}{1-y} = wx + b$$

那么为什么和逻辑回归是等价的呢？ 简单化简一波：

$$\ln \frac{y}{1-y} = wx + b$$
$$\frac{y}{1-y} = e^{wx+b}$$
$$y = e^{wx+b} - y * e^{wx+b}$$
$$y\left(1 + e^{wx+b}\right) = e^{wx+b}$$
$$y = \frac{e^{wx+b}}{1+e^{wx+b}} = \frac{1}{1+e^{-(wx+b)}}$$

这样就得到了逻辑回归的那个式子。

那么我们就来看看这个逻辑回归或者对数几率回归了， 我们知道线性回归，使用 w x + b去拟合数值 y, 而对数几率回归就是用 w x + b 去拟合一个几率 $\ln \frac{y}{1-y}$ 。

原理说的差不多了，那就实践吧 训练一个逻辑回归模型，先说一下机器学习模型训练的步骤：



1. 数据模块（数据采集，清洗，处理等）
2. 建立模型（各种模型的建立）
3. 损失函数的选择（根据不同的任务选择不同的损失函数），有了loss就可以求取梯度
4. 得到梯度之后，我们会选择某种优化方式去进行优化
5. 然后迭代训练

后面建立各种模型，都是基于这五大步骤进行， 这个就相当于一个逻辑框架了。

下面就基于上面的五个步骤，看看Pytorch是如何建立一个逻辑回归模型，并分类任务的。我们下面一步一步来：

1. 数据生成
   这里我们使用随机生成的方式，生成2类样本（用0和1表示）， 每一类样本100个，每一个样本两个特征。

```
"""数据生成"""
torch.manual_seed(1)

sample_nums = 100
mean_value = 1.7
bias = 1

n_data = torch.ones(sample_nums, 2)
x0 = torch.normal(mean_value*n_data, 1) + bias   # 类别0
y0 = torch.zeros(sample_nums)    # 类别0，  数据shape=(100
x1 = torch.normal(-mean_value*n_data, 1) + bias    # 类别
y1 = torch.ones(sample_nums)     # 类别1   shape=(100, 1)
```

2. 建立模型

这里我们使用两种方式建立我们的逻辑回归模型，一种是Pytorch的sequential方式，这种方式就是简单，易懂，就类似于搭积木一样，一层一层往上搭。 另一种方式是继承nn.Module这个类搭建模型，这种方式非常灵活，能够搭建各种复杂的网络。

```
"""建立模型"""
class LR(torch.nn.Module):
    def __init__(self):
        super(LR, self).__init__()
        self.features = torch.nn.Linear(2, 1)  # Linea
        self.sigmoid = torch.nn.Sigmoid()

    def forward(self, x):
        x = self.features(x)
        x = self.sigmoid(x)

        return x
```

算法原理：**前向传播**：实现逻辑回归函数

```
y_pred = lr_net(train_x)
```

- **线性变换**：模型首先对输入数据 `train_x` 进行线性变换，即通过权重矩阵 WW 和偏置项 bb 计算出线性输出：

  z=W·x+b

  这里，x 是输入特征，W 是模型的权重矩阵，b 是偏置项。

- **激活函数**：然后，线性输出 z 被传递到 Sigmoid 函数中，将其转换为一个概率值：

$$y = f(wx + b)$$
$$f(x) = \frac{1}{1 + e^{-x}}$$

`Sigmoid` 函数将线性输出映射到 `(0, 1)` 区间，这样可以解释为样本属

另外一种方式，Sequential的方法：

```
lr_net = torch.nn.Sequential(
    torch.nn.Linear(2, 1),
    torch.nn.Sigmoid()
)
```

3. 选择损失函数
   这里我们使用二分类交叉熵损失

```
"""选择损失函数"""
loss_fn = torch.nn.BCELoss
```

与真实标签之间的差异。

- 其公式为：

$$L = -\left(y \log(p) + (1 - y) \log(1 - p)\right)$$

其中：

- y 是真实标签（0 或 1）。
- p 是模型预测的概率（即 y_pred 的输出）。

交叉熵损失有以下几个重要特性：

- **惩罚错误预测**：当模型预测的概率 pp 与真实标签 yy 相差较大时，交叉熵损失会急剧增大。例如，如果真实标签 y=1y=1，但模型预测的概率 p=0.1p=0.1，则损失将非常大，促使模型显著调整参数
- **平滑且凸的优化空间**：交叉熵损失函数具有平滑且凸的形状，这使得基于梯度的优化方法（如 SGD）能够更容易找到全局最优解，而不会陷入局部极小值

因此，通过最小化交叉熵损失，模型会逐渐调整其参数，使得预测概率 p 更接近真实标签 y，从而提高分类准确率。

4. 选择优化器

- 优化器的知识比较复杂，后面也是单独会介绍
- 这里我们就用比较常用的SGD优化器。实现随机梯度下降（Stochastic Gradient Descent），用来调整神经网络中的参数，以最小化损失函数。
- 关于这些参数先看一看。这也就是为什么要系统学习一遍Pytorch的原因， 有的知识互相关联，不必完全清楚。学完一遍之后会更加清楚前面的一些问题

```
"""选择优化器"""
lr = 0.01
optimizer = torch.optim.SGD(lr_net.parameters(), lr=lr
```

5. 迭代训练模型

- 这里就是我们的迭代训练过程了， 基本上也比较简单，在一个循环中反复训练
- 前向传播-->计算loss-->计算梯度(反向传播)-->更新参数-->梯度清零。

```
"""模型训练"""
for iteration in range(1000):

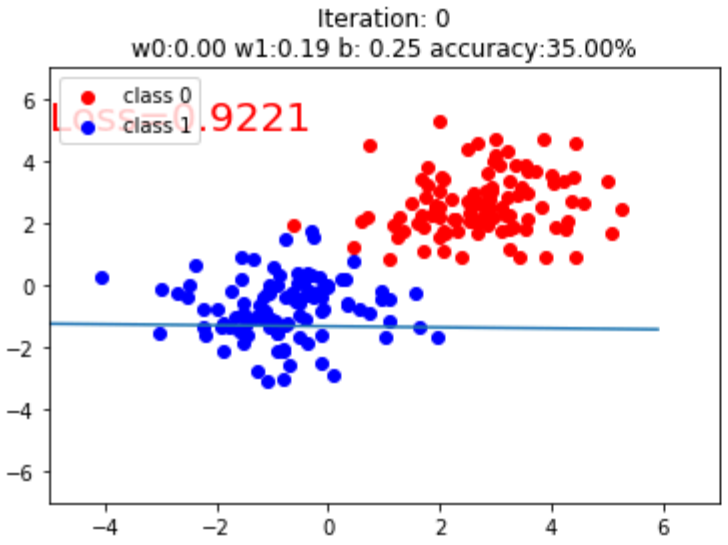
    # 前向传播
    y_pred = lr_net(train_x)

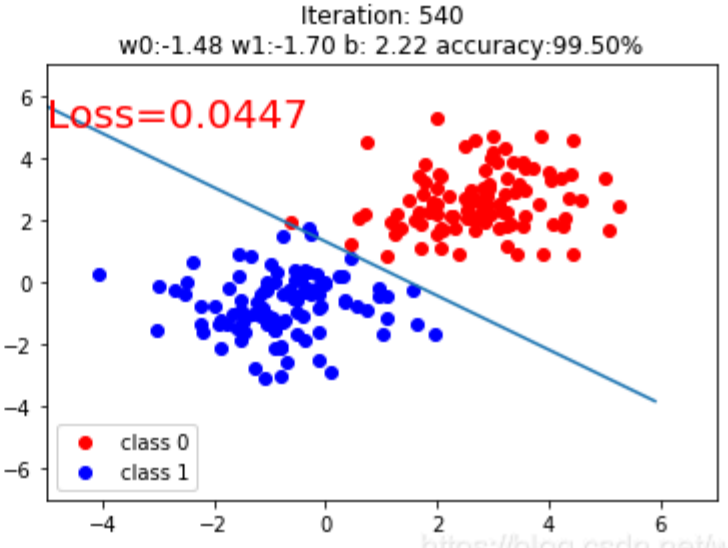
    # 计算loss
    loss = loss_fn(y_pred.squeeze(), train_y)

    # 反向传播
    loss.backward()

    # 更新参数
```

别看这么多，后面都是绘图的过程，不是重点。重点训练就是前面的那几步。 我们可以看看结果：





https://blog.csdn.net/wuzhongqiang

这就是我们的逻辑回归模型进行二分类的问题了。

提问： 在Pytorch中，为什么需要手动清除梯度（如使用`optimizer.zero_grad()`）？

- A. 因为Pytorch会自动清除梯度
- B. 因为每次反向传播时，梯度会累加，不清除会导致错误结果
- C. 因为每次反向传播都会重置所有参数
- D. 因为不清除梯度会影响模型的预测准确性

答案： B

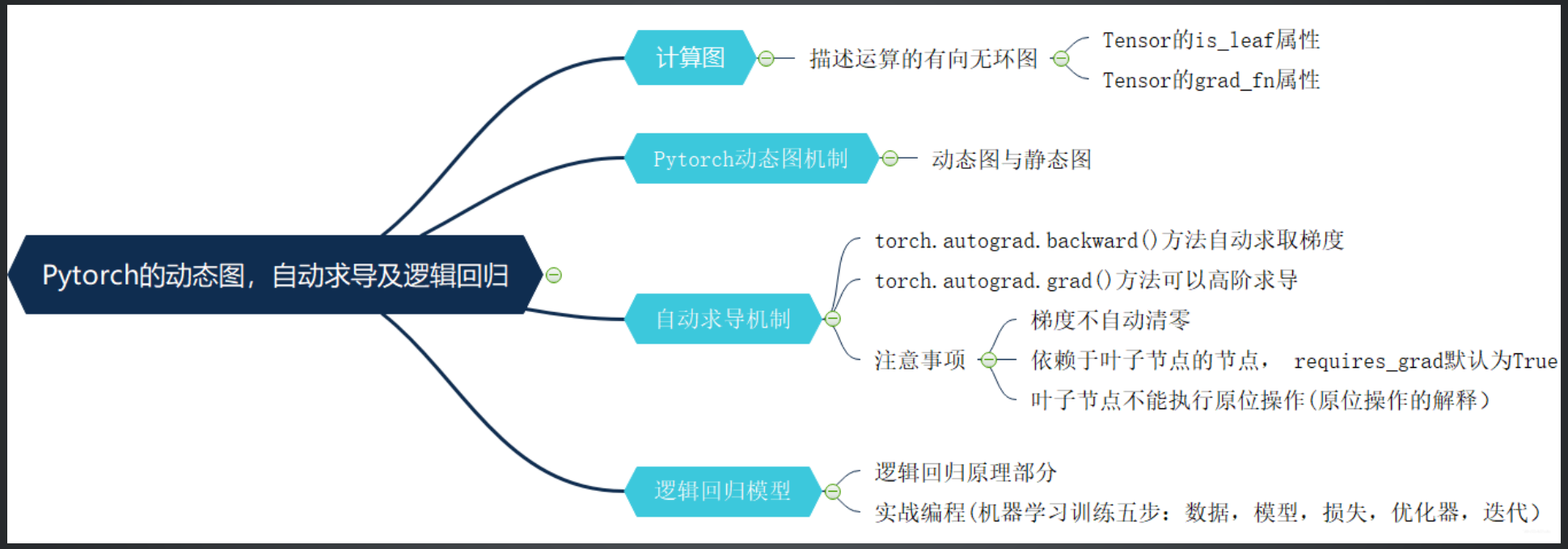
讲评：

在Pytorch中，每次反向传播时，梯度不会自动清零，而是累加。因此，如果不手动清除梯度（如使用optimizer.zero_grad()），梯度值会不断累加，导致错误结果，尤其是在训练神经网络时

5. 总结梳理

- 首先基于前面的张量的知识我们又更进一步，学习了计算图的机制
    - 计算图说白了就是描述运算过程的图， 有了这个图梯度求导的时候非常方便。
- 然后学习了Pytorch的动态图机制，区分了一下动态图和静态图。
- 然后学习了Pytorch的自动求导机制，认识了两个比较常用的函数torch.autograd.backward()和torch.autograd.grad()函数
    - 关于自动求导要记得三个注意事项：
        - 梯度手动清零，
        - 叶子节点不能原位操作，
        - 依赖于叶子节点的节点默认是求梯度。
- 最后我们根据上面的所学知识建立了一个逻辑回归模型实现了一个二分类的任务，

下面依然是一张导图把这次学习的知识整理出来：



下一节学习Pytorch的数据读取机制DataLoader和Dataset 😉